EXPERIMENT-10

Applying Performance Optimization Techniques in Zoho Creator

Aim

To apply performance optimization techniques such as indexing, subforms, and query tuning in the Zoho Creator Insurance Claim application to improve data retrieval efficiency and application performance.

Overview:

1. Indexing: Indexing helps improve the speed of searching and retrieving records by enabling efficient access to frequently searched data. Fields such as Policy_Number and Claim_Status may be important for frequent searches and filtering. Appropriate indexing can reduce search time when supported by the platform. However, Zoho Creator may manage indexing internally, and manual indexing options may not be available for every field.

2. Subforms: Subforms allow multiple related entries to be stored within a single parent record. In the Insurance Claim application, a subform can be used to maintain multiple expense details associated with one claim. For example, Expense_Type, Description, and Expense_Amount can be recorded as separate entries. This improves data organization and makes it easier to manage related information without creating unnecessary independent forms.

3. Query Tuning: Query tuning involves improving database queries so that they retrieve only the required records and process data efficiently. In the Insurance Claim application, criteria-based queries can retrieve a particular insurance policy or calculate the total approved claim amount for a selected policy. Filtering records by Policy_Number and Claim_Status avoids processing unrelated records and improves query efficiency.

4. Avoiding Repeated Queries: Repeated database queries can increase processing overhead and slow down workflows. Reusing previously retrieved records and calculated values helps reduce unnecessary database operations. In the claim validation workflow, retrieving the selected policy once and reusing its coverage information can help improve the efficiency of processing.

5. Error Handling and Validation: Proper error handling ensures that workflows continue to produce correct results when queries return no matching records. For example, if no approved claims exist for a policy, the total approved claim amount should be treated as zero. Validating claim amounts and checking the remaining coverage also help maintain data accuracy and prevent invalid submissions.

6. Performance Testing: Testing is necessary to verify that optimization techniques work correctly. The Insurance Claim application can be tested using valid claims, invalid claim amounts, and policies with different remaining coverage values. Query execution time and application responsiveness can be compared before and after changes when reliable measurements are available.

Procedure:

1. **Open the Application:** Log in to Zoho Creator and open the existing Insurance Claim application in edit mode. Identify the Customer Details, Insurance Policy, and Insurance Claim forms that are used to store and manage application data.

2. **Identify Frequently Accessed Fields:** Examine the fields used frequently in searches, reports, and workflows. Fields such as Policy_Number, Claim_Status, and Customer lookup fields are important because they are used to retrieve and filter records.

3. **Explore Indexing Options:** Open the Insurance Policy form and select the Policy_Number field. Inspect its field properties and settings to check whether indexing or search optimization is available. If supported, configure the appropriate option for frequently searched fields. Save the changes if any are made.

4. **Review Existing Workflows:** Navigate to the Workflows section and open the Validate_Claim_Policy_Customer workflow. Examine the existing Deluge script to understand how policy records are retrieved and how claim amounts are validated against the remaining policy coverage.

5. **Optimize Policy Record Retrieval:** Use criteria-based queries to retrieve only the required policy record instead of retrieving all policy records. For example, the query `Insurance_Policy[ID == input.Policy_Number]` retrieves the policy record matching the selected policy ID. This reduces unnecessary record retrieval.

6. **Optimize Approved Claim Queries:** Filter insurance claims using Policy_Number and Claim_Status before calculating the total approved amount. This ensures that only approved claims associated with the selected policy are included in the calculation, avoiding unnecessary processing of unrelated records.

7. **Handle Empty Query Results:** Check whether the sum of approved claim amounts returns a null value. If no matching records exist, initialize the total approved amount to zero. This prevents calculation errors when determining the remaining policy coverage.

8. **Avoid Repeated Queries:** Examine the Deluge workflow for repeated queries that retrieve the same information. Reuse previously retrieved records and calculated values wherever possible. Avoid unnecessary database operations, especially when processing multiple claim records.

9. **Explore Subform Functionality:** Open the Insurance Claim form in the application builder and examine the available field types. A subform can be used to store multiple related entries within a single claim record, such as individual expenses or supporting cost details.

10. **Configure a Subform if Required:** Create a subform named Claim_Expense_Details, if appropriate for the application. Add fields such as Expense_Type, Description, and Expense_Amount. Save the form and enter sample expense entries to verify that multiple related items can be stored under one insurance claim.

11. **Test Valid Claim Submission:** Submit a test claim with a positive claim amount that does not exceed the remaining policy coverage. Verify that the workflow retrieves the correct policy, calculates the approved claims total, and accepts the valid submission.

12. **Test Invalid Claim Scenarios:** Test zero and negative claim amounts, as well as a claim amount that exceeds the remaining policy coverage. Verify that the workflow rejects invalid submissions and displays the appropriate validation messages.

13. **Evaluate the Optimization:** Review the query results, claim validation, remaining coverage calculations, and subform functionality, if implemented. Compare execution times before and after changes when measurable timing information is available. Confirm that the existing application workflows continue to operate correctly.

Result:

Thus, performance optimization techniques such as query tuning, handling null values, avoiding repeated database queries, and exploring subforms were studied and applied where applicable in the Zoho Creator Insurance Claim application. These techniques help improve data retrieval efficiency, data organization, and application maintainability.